CLAUDE SONNET 5

Does it win the actual workload?

A rigorous investigation of capability, benchmark methodology, agent scaffolding, reliability and cost per successful task

Research cut-off

12 July 2026. The report separates the launch-time comparison on 30 June from the current market after GPT-5.6 became generally available on 9 July. Dynamic leaderboards may change after this date.

Prepared for Jose Parreño Garcia
Senior Data Science Lead

Evidence-led research dossier - not a vendor benchmark recap

How to read this report

This report starts with the simple question - what changed from Sonnet 4.6? - and gradually adds the complications that determine whether a model is useful in practice. The central distinction is between a raw model capability claim and the performance of a complete system containing prompts, tools, a coding agent, retries, compaction, caching and infrastructure.

Every important claim is followed by clickable source identifiers. The source register in Appendix B records the publication date, evidence type, confidence tier and limitations. Vendor documentation is treated as authoritative for specifications, but never as independent proof that the vendor's model is better.

A note about charts

The charts in this PDF are original reconstructions from reported numbers; no vendor chart has been copied. Where a source contains a useful interactive chart, the relevant section includes a “source chart to inspect” note and a clickable citation. This lets you check the original axes, footnotes and effort settings rather than trusting a screenshot stripped of context.

Contents

1. Executive verdict

The central finding

Sonnet 5 is a real upgrade, not merely a relabelled Sonnet 4.6. Its clearest practical advance is sustained agentic work: repository coding, terminal execution, tool use, self-checking and multi-step professional tasks. The evidence does not support the broader claim that it is the best model for every workload. Its strongest settings often spend enough tokens and turns to erase the apparent price advantage of the Sonnet tier.

Anthropic's strongest evidence is concentrated in agentic settings. On its system-card evaluation, Sonnet 5 rises from 58.1% to 63.2% on SWE-bench Pro, from 67.0% to 80.4% on Terminal-Bench 2.1, from 15.1% to 38.8% on FrontierCode and from 5.3% to 13.5% on AutomationBench. Those are not small changes. They point to a model that persists longer, uses tools more effectively and reaches a working end state more often. *Sources:* [A02]

The economic conclusion is less flattering. Artificial Analysis found that Sonnet 5 at maximum effort used roughly 40% more output tokens per Intelligence Index task than Sonnet 4.6 and around three times as many agent turns on two knowledge-work evaluations. At standard post-promotion pricing, its estimated $2.29 per Intelligence Index task was about twice Sonnet 4.6 and around 15% above Opus 4.8, despite a cheaper token tariff. *Sources:* [B15]

A second independent signal comes from CursorBench 3.2. Sonnet 5 Max scores 61.5%, but at an average $6.45 per task and 86 agent steps. Opus 4.8 Max scores slightly higher at 62.3%, costs slightly less per task at $5.77 and takes 44 steps. GPT-5.6 Sol High, released after Sonnet 5, scores 63.5% at $2.79 and 32 steps. The conclusion is not that CursorBench settles the market; it is that Sonnet 5's maximum-effort mode is not automatically the rational economic default. *Sources:* [B14], [P08]

Hypothesis assessment

Hypothesis	Verdict	Why
H1: coding and agentic software engineering are the clearest gain	Supported, with nuance	Large official gains on Terminal-Bench, FrontierCode and AutomationBench; independent CursorBench and practitioner reports broadly confirm stronger building and persistence. Code review recall can regress.
H2: stronger scores may lose on cost per correct task	Supported	AA finds higher task cost than 4.6 and Opus 4.8 at standard pricing. CursorBench shows medium/high effort often dominates max on economic efficiency.
H3: independent evidence is more mixed than the launch chart	Supported	Independent same-harness results still show Sonnet 5 as strong, but not dominant; vendor charts use varied tools, contexts, trials and scaffolds.
H4: better than 4.6 does not mean best available	Supported	At launch it was near the frontier. By 12 July, GPT-5.6 and Fable led several independent indices and coding-agent comparisons.
H5: product and scaffold matter heavily	Strongly supported	The system card mixes mini-SWE-agent, Cursor production harness, Anthropic internal agents and tool-rich research harnesses. RuBench even found silent product-level model substitution.

Direct answers to the fourteen questions

Question	Answer
1. What is genuinely better than 4.6?	Long-horizon coding, terminal work, tool use, self-verification, some professional workflows, chart/document reasoning and safety behaviour.
2. Where is it practically meaningful?	Repository-level implementation and multi-step agents, especially when failure is expensive enough to justify medium-to-high effort.
3. Where is the gain mainly visible in Anthropic evidence?	Writing, broad reasoning and long-context quality have less independent task-specific confirmation. The nominal context window did not grow.
4. Do independent results confirm coding claims?	Broadly yes for building and agent persistence; no for every coding subtask. CodeRabbit reports better precision but lower bug-catching recall than 4.6.
5. Is it the strongest coding model?	One of the strongest. It is not consistently first after GPT-5.6 and Fable 5, and the answer changes by agent harness and budget.
6. Is it best after cost?	Usually not at maximum effort. Medium or high effort can be sensible; several competitors offer cheaper expected success in specific harnesses.
7. Model or scaffold?	Both. The model improved, but a benchmark score belongs to the model-agent-tool-infrastructure system.
8. Writing, reasoning, long context?	Reasoning improves on several tests; writing evidence is insufficient; nominal context remains 1M and the new tokenizer can reduce effective text capacity.

Question	Answer
9. Better-value competitors?	GPT-5.6 Sol High in current coding-agent tests; Gemini Flash for speed; DeepSeek, Kimi and some open models for lower-cost workloads, accepting lower aggregate quality.
10. Who should upgrade?	Claude Code users doing multi-file or multi-step work, and teams whose current failure/retry cost is high.
11. Who should remain on 4.6?	Stable, latency-sensitive, high-volume tasks where 4.6 already passes and extra agentic persistence becomes overwork.
12. Who should use another provider?	Teams optimising for current frontier coding, very low cost, very low latency, or an ecosystem-specific agent scaffold.
13. What remains missing?	Large independent production studies, writing preference tests, repeated-run latency distributions, and clean raw-model comparisons without scaffold changes.
14. “Only good at beating 4.6”?	Partly disproved. It beats or matches strong external models on some tasks, but its most reliable story remains a large Sonnet-tier upgrade rather than universal leadership.

2. Research method and evidence hierarchy

The report uses an explicit evidence hierarchy. Primary product documentation answers questions such as price, context length and API behaviour. Raw benchmark repositories and papers explain what a benchmark actually measures. Independent same-harness evaluations carry more weight for comparison. Company case studies are useful when they reveal methodology and inconvenient results. Community comments are treated as leads, not conclusions.

Six categories that must not be collapsed

Category	Interpretation
Raw model capability	What the model can do under a specified prompt and tool interface. Even here, “raw” usually includes a sampling policy and reasoning budget.
Agent or product performance	Outcome of the model plus Claude Code, Codex, Cursor, browser tools, retries, memory, compaction and orchestration.
Economic efficiency	Quality achieved per unit of total spend, including failed runs, cached context, reasoning, tools and engineering overhead.
Reliability	How often the system succeeds across repeated trials. A high pass@k can hide a weak pass@1.
User preference	Whether humans prefer an output. This can reward clarity or style without proving objective correctness.
Benchmark performance	A result on a fixed task distribution. It is useful evidence, not a guarantee that the same ranking transfers to production.

The most important methodological rule is simple: do not compare scores produced under materially different conditions as if they were one league table. Sonnet 5's official results include five-trial averages, a 10-million-token BrowseComp budget with compaction, a 100-action computer-use limit, a repaired Toolathlon environment and several different agent harnesses. Those details are not footnote trivia; they define the treatment being measured. *Sources:* [A02], [A08], [A09], [A10]

Launch-time versus current-time comparison

Sonnet 5 launched on 30 June 2026. GPT-5.6 became generally available on 9 July. Therefore, two questions need separate answers: Was Anthropic's launch positioning reasonable against the models available at launch? And is Sonnet 5 the best choice now? A later competitor can change the second answer without making the original improvement fictitious. *Sources:* [A01], [P08]

3. What Anthropic claims

3.1 Product position and specifications

Anthropic calls Sonnet 5 its most agentic Sonnet and positions it for advanced coding, long-running agents and professional work. It is available in Claude, Claude Code and the API under the model identifier claude-sonnet-5. The nominal context window remains 1 million tokens and the synchronous maximum output remains 128,000 tokens. *Sources:* [A01], [A03], [A06]

The launch promotion prices input at $2 per million tokens and output at $10 until 31 August 2026. Standard pricing from 1 September is $3 and $15, identical to Sonnet 4.6's list tariff. Prompt-cache hits receive a 90% discount relative to ordinary input; five-minute cache writes carry a 25% premium. Batch processing can reduce eligible API costs by 50%. *Sources:* [A01], [A04]

The tokenizer caveat

Anthropic says the new tokenizer maps the same text to roughly 1.0-1.35 times as many tokens, with about 30% being a useful planning estimate. Therefore, equal per-token pricing does not imply equal request cost. At a 1.30 ratio, a nominal 1M context holds only about 769,000 old-token equivalents of the same text. That is an approximation, not a universal conversion. *Sources:* [A03], [A04]

3.2 Behaviour and migration changes

Adaptive thinking is enabled by default. Sonnet 5 supports low, medium, high, xhigh and max effort, with high as the API and Claude Code default. Anthropic warns that effort labels are not directly comparable across generations: Sonnet 5 medium can resemble Sonnet 4.6 high in observed thinking length, while Sonnet 5 high can resemble the previous maximum. Fair benchmarking should match observed compute or token use, not merely the name printed on the dial. *Sources:* [A03], [A05]

Manual extended thinking through budget_tokens is removed. Non-default sampling settings such as temperature, top_p and top_k can return a 400 error. Thinking and final answer share max_tokens, which means an aggressive reasoning setting can consume output budget that a long final deliverable also needs. Migration therefore requires retesting token limits, latency and prompt instructions rather than swapping a model string and hoping for the best. *Sources:* [A03], [A05]

Anthropic also describes Sonnet 5 as more literal and more proactive about using tools, checking its work and pursuing an end state. This is an advantage for open-ended agents but can become over-engineering on tiny tasks. A model that writes tests, adds helpers and performs another inspection pass may improve reliability while simultaneously increasing latency and billable context. *Sources:* [A05], [P06]

3.3 Official benchmark summary

Benchmark	S5	S4.6	Absolute	Relative	Area
SWE-bench Pro	63.2	58.1	+5.1 pp	+8.8%	Coding agent
Terminal-Bench 2.1	80.4	67.0	+13.4 pp	+20.0%	Terminal agent
BrowseComp (single agent)	84.7	76.2	+8.5 pp	+11.2%	Agentic search
HLE - no tools	43.2	34.6	+8.6 pp	+24.9%	Reasoning/knowledge
HLE - with tools	57.4	46.8	+10.6 pp	+22.6%	Tool-assisted knowledge
OSWorld-Verified	81.2	78.5	+2.7 pp	+3.4%	Computer use
FrontierCode v1	38.8	15.1	+23.7 pp	+157.0%	Repository coding
AutomationBench	13.5	5.3	+8.2 pp	+154.7%	Enterprise automation
Legal Agent Benchmark - all pass	8.9	8.0	+0.9 pp	+11.3%	Legal agent
HealthBench Professional	57.8	44.2	+13.6 pp	+30.8%	Professional health

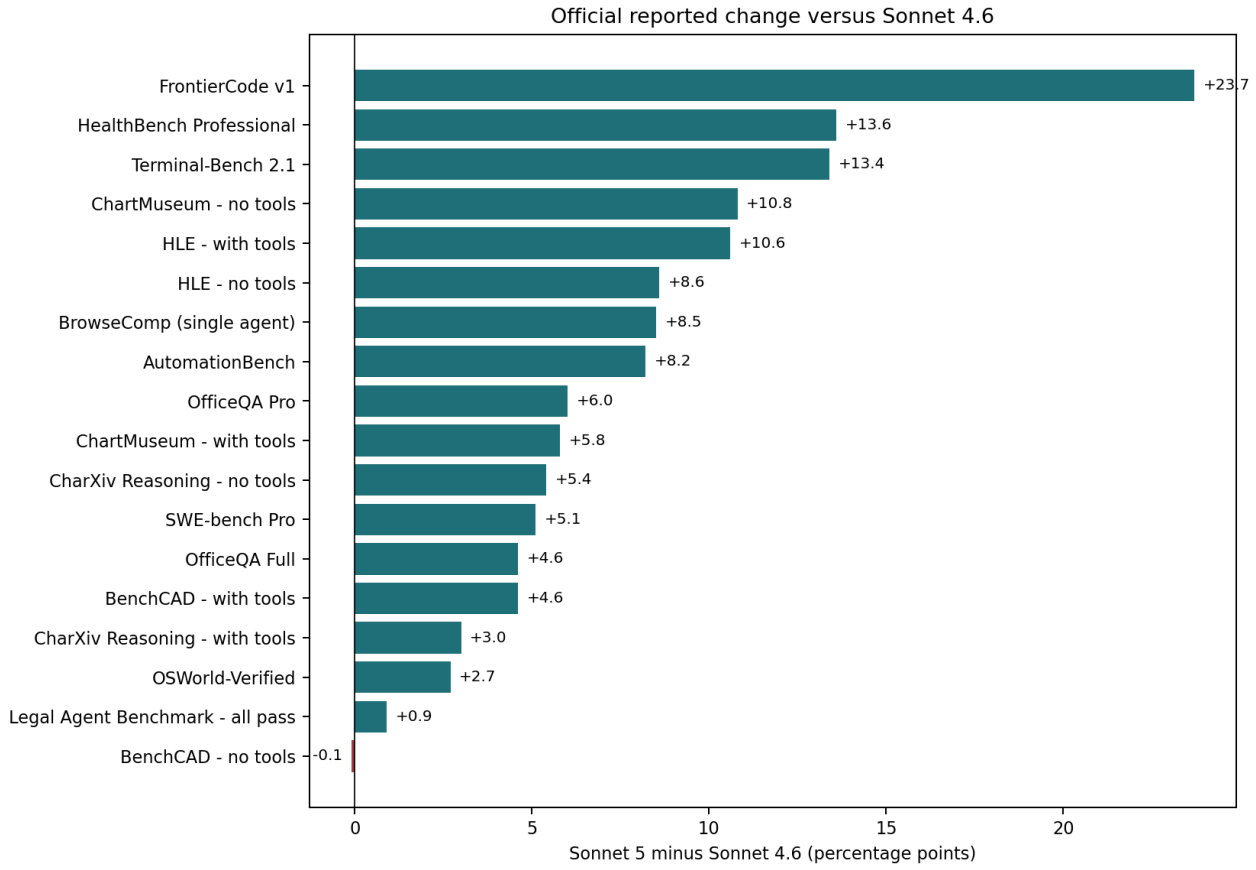

Figure 1. Officially reported percentage-point changes from Sonnet 4.6 to Sonnet 5.

Reconstructed from the Sonnet 5 System Card. Percentage-point deltas should not be confused with relative percentage improvement. Source chart and footnotes: [A02].

The official pattern is not “a little better everywhere”. It is a mixture of very large agentic gains, modest computer-use gains, strong improvements on some professional tasks and one essentially flat no-tool CAD result. That variation already contradicts both simplistic hype and simplistic dismissal. *Sources:* [A02]

3.4 Safety and reliability claims

Anthropic reports a lower overall rate of undesirable behaviour than Sonnet 4.6, stronger prompt-injection resistance and lower hallucination and sycophancy. It also emphasises that Sonnet 5 remains less capable in advanced cyber tasks than its Opus-class models. These are useful pre-deployment signals, but they should be interpreted as measured robustness under defined tests, not immunity once the model is connected to files, terminals and credentials. *Sources:* [A01], [A02]

The distinction is already visible in emerging evidence. A July proof of concept reportedly manipulated autonomous Claude Code configurations, including Sonnet 5, through prompt injections hidden in repository content. This does not negate Anthropic's measured improvement; it shows that a lower attack-success rate on a benchmark can coexist with exploitable workflows in a different threat model. *Sources:* [C02]

4. Benchmark methodology audit

The benchmark forensic rule

A score without its harness, effort, tools, attempts, context budget and scoring rule is an incomplete observation. When any of those differ, the models may not be receiving the same treatment.

4.1 Audit table: what the headline benchmarks really measure

Benchmark	Measures	Tasks/data	Tools/scaffold	Scoring/repeats	Contamination/ reproducibility	Production relevance	Sources
SWE-bench Verified / Pro	Fix real GitHub issues and pass tests	Verified: 500 public human-checked tasks; Pro is harder, multi-file and designed to reduce leakage	Agent scaffold, shell and repository access	Average over five trials in Anthropic card	Public repositories and fixes create contamination risk	Good proxy for repository repair, not greenfield design or code review quality	[A02] [B01] [B02] [B03]
Terminal-Bench 2.1	Complete command-line tasks in containers	89 tasks after 28 fixes in v2.1	Anthropic used mini-SWE-agent; 1x timeout, 3x memory ceiling	Five attempts per task, 445 trials	Lower contamination concern than old code benchmarks, but tasks are public	Strong for terminal execution; sensitive to timeout and harness behaviour	[A02] [B05]
FrontierCode v1	Implement real pull requests with held-out tests and rubric criteria	150 tasks; system-card description	Agentic coding environment; effort-cost frontier	Reported score and average cost per task	Freshness stronger than classic SWE-bench, but full external reproduction is limited	Relevant to repository engineering; cost curve is especially useful	[A02]
CursorBench 3.2	Ambiguous multi-file coding from real Cursor sessions	Private tasks; current version adds instruction following and advanced tools	Cursor production agent harness	Single published aggregate plus cost/tokens/steps	Private tasks reduce contamination but prevent independent audit; Grok has disclosed advantage	One of the best available product-style coding comparisons	[B14]
BrowseComp	Find difficult, verifiable information on the web	1,266 questions with short answers	Web search, fetch, code, programmatic tools; 10M total-token budget and compaction for Sonnet	Single and multi-agent results	Question sources are blocklisted in Anthropic run; benchmark distribution is intentionally difficult	Measures persistence and retrieval, not normal research writing or source synthesis	[A02] [B06]
Humanity's Last Exam	Answer extremely difficult questions across 100+ subjects	2,500 questions; public and held-out components	No-tool and tool-rich variants; Anthropic uses web, code and up to 1M tokens	Average over trials; exact scoring by benchmark	Contamination screened with blocklists and transcript review	Useful frontier reasoning signal, weak direct proxy for business workflows	[A02] [B08]
OSWorld-Verified	Operate real desktop applications	361 evaluated tasks after exclusions	1080p desktop, up to 100 actions, model sees screenshots and acts	Pass@1 averaged over five runs	UI state and infrastructure drift are major sources of variance	Relevant to computer use, but not a raw visual-reasoning test	[A02] [B07]
GDPval-AA v2	Produce professional deliverables across occupations	220-task public gold subset of broader GDPval; 44 occupations / 9 industries	Stirrup shell-and-web agent	Blind pairwise model-graded Elo	Tasks are richer but judgement and grader models matter	Good knowledge-work proxy; still an agent-system evaluation	[A02] [B09] [B10]
Toolathlon	Complete long multi-app workflows	108 tasks, 604 tools, 32 applications	Internal repaired harness pins dependencies	Pass@1, Pass@3 and Pass^3; about 20+ turns	A quarter of published tasks reportedly need repairs; internal offset estimated around +3	Excellent for reliability and tool orchestration when methodology is disclosed	[A02] [B11]
OfficeQA	Answer questions over large public document corpora	246 Full and 133 Pro; roughly 89,000 pages	Document conversion/search tools; representation strategy is crucial	Exact-match style QA; output-limit failures count wrong	Public corpus; parsing differences dominate some results	Strong long-document retrieval test, not general long-context comprehension	[A02] [B12]
AutomationBench	Build workflows across enterprise applications	Private held-out tasks	API discovery and tool execution	Deterministic end-state grading	Low contamination, lower reproducibility	Highly relevant to enterprise agents; still early and low absolute success	[A02] [B13]
Legal Agent Benchmark	Perform complex legal-agent tasks with many criteria	Public and Harvey held-out sets	Legal research/drafting agent	All-pass and mean criterion pass	Company-held subset limits independent reproduction	All-pass is strict and practically meaningful, but can mask average-criterion regression	[A02] [P04] [P05]

4.2 Pass@1, pass@k and repeated-run reliability

Pass@1 asks whether one run succeeds. Pass@3 often asks whether at least one of three runs succeeds. Those are different product promises. If a system gets one success in three, it may look useful under pass@3 while still failing twice for every user who only runs it once. Pass^3 is stricter: it asks whether all three runs succeed and therefore rewards consistency.

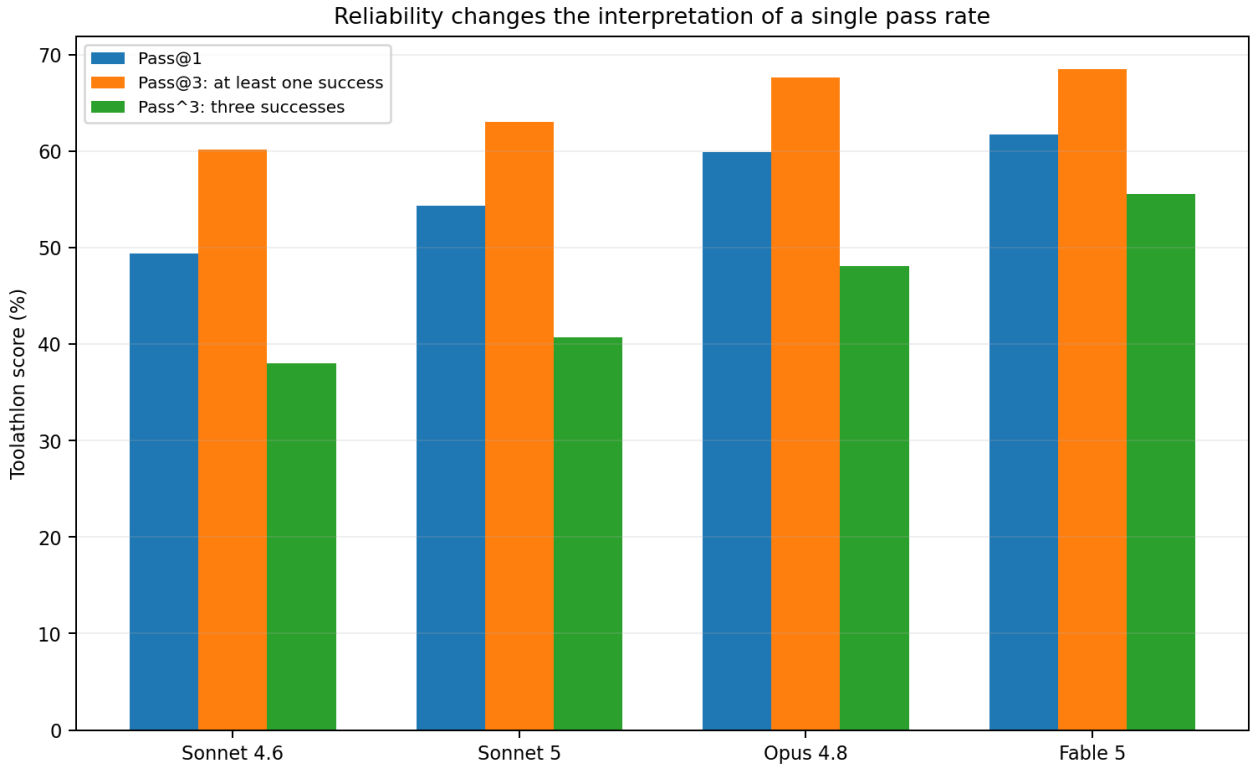

Figure 2. Toolathlon shows why reliability needs more than one number.

Sonnet 5 improves over 4.6 on all three measures, but also uses 26.0 average turns versus 16.5. Source table: [A02].

On Toolathlon, Sonnet 5 moves from 49.4% to 54.3% pass@1 and from 38.0% to 40.7% pass^3. That is genuine improvement, but the reliability gain is smaller than the raw increase in activity: average turns rise by about 58%. A system that works more often and works much longer is better in capability terms, but the economic conclusion remains open. *Sources:* [A02]

4.3 Contamination and freshness

Public coding benchmarks face an unusual problem: the model may have encountered the repository, issue or final patch during training. A 2025 diagnostic study found models could identify buggy file paths from issue descriptions alone far better on SWE-bench repositories than on similar outside repositories, and could reproduce ground-truth functions with higher verbatim overlap. This is evidence of possible memorisation, not proof that every benchmark success is fake. *Sources:* [B03]

Fresh or continuously refreshed benchmarks reduce that risk. RuBench uses fixes that postdate the evaluated models' training cut-offs, while SWE-rebench and SWE-bench Live are designed around newer tasks. The trade-off is smaller samples, more expensive maintenance and less stable longitudinal comparison. Scientific hygiene rarely produces a conveniently permanent leaderboard. *Sources:* [B04], [B20]

4.4 Infrastructure and scaffold effects

Anthropic's own engineering work argues that infrastructure configuration can move agentic coding scores by more than typical leaderboard margins. Timeouts, memory ceilings, command execution, repository setup and even how the agent waits for a shell process can change completion rates. Sonnet 5's Terminal-Bench result uses mini-SWE-agent because Anthropic found Terminus-2 produced 2.7 times more timeouts at xhigh effort. *Sources:* [A02], [A09]

This is not an excuse to ignore benchmarks. It is a reason to name the experimental unit correctly. “Sonnet 5 scored 80.4” is shorthand for “Sonnet 5, at xhigh effort, in mini-SWE-agent, with the stated resource policy and five attempts per task, averaged 80.4 on Terminal-Bench 2.1”. The longer sentence is less catchy and much more scientific. *Sources:* [A02], [A10]

4.5 Benchmarks deliberately not converted into comparisons

This report does not quote Sonnet 5 numbers for LiveCodeBench, LiveBench, Aider Polyglot, OpenHands, WebArena, GPQA, MMLU-Pro or classic needle-in-a-haystack retrieval. By the research cut-off, I did not find a sufficiently matched Sonnet 5 versus Sonnet 4.6

versus current-competitor run under the same harness, effort, tool and attempt settings. Combining isolated vendor or community numbers would create false precision rather than additional evidence.

These benchmarks are not useless. LiveCodeBench can help with fresh code-generation questions; OpenHands can compare models inside one agent scaffold; WebArena tests browser interaction; GPQA and MMLU-Pro probe academic knowledge; and retrieval tests can expose context failures. They should enter a procurement decision only when the exact model snapshots and evaluation treatments are aligned. Absence from this report means insufficient comparable evidence, not a negative score.

5. Sonnet 5 versus Sonnet 4.6

5.1 Task-by-task change

Benchmark	4.6	5	Delta	Relative	Practical label
SWE-bench Pro	58.1	63.2	+5.1 pp	+8.8%	Moderate
Terminal-Bench 2.1	67.0	80.4	+13.4 pp	+20.0%	Large
BrowseComp (single agent)	76.2	84.7	+8.5 pp	+11.2%	Moderate
HLE - no tools	34.6	43.2	+8.6 pp	+24.9%	Moderate
HLE - with tools	46.8	57.4	+10.6 pp	+22.6%	Large
OSWorld-Verified	78.5	81.2	+2.7 pp	+3.4%	Small
FrontierCode v1	15.1	38.8	+23.7 pp	+157.0%	Large
AutomationBench	5.3	13.5	+8.2 pp	+154.7%	Moderate
Legal Agent Benchmark - all pass	8.0	8.9	+0.9 pp	+11.3%	Small
HealthBench Professional	44.2	57.8	+13.6 pp	+30.8%	Large
OfficeQA Full	68.7	73.3	+4.6 pp	+6.7%	Moderate
OfficeQA Pro	53.4	59.4	+6.0 pp	+11.2%	Moderate
ChartMuseum - no tools	59.3	70.1	+10.8 pp	+18.2%	Large
ChartMuseum - with tools	80.9	86.7	+5.8 pp	+7.2%	Moderate
CharXiv Reasoning - no tools	71.6	77.0	+5.4 pp	+7.5%	Moderate
CharXiv Reasoning - with tools	85.3	88.3	+3.0 pp	+3.5%	Small
BenchCAD - no tools	26.7	26.6	-0.1 pp	-0.4%	No clear change
BenchCAD - with tools	32.7	37.3	+4.6 pp	+14.1%	Moderate

The practical label is deliberately conservative. A 10-point benchmark gain can be large even when it is not statistically significant, and a 3-point gain can be highly valuable if it moves a production process across a reliability threshold. The table is a descriptive triage, not a claim of universal business impact.

5.2 Coding and repository engineering: large improvement

The strongest case is repository work. FrontierCode more than doubles from 15.1% to 38.8%; Terminal-Bench rises by 13.4 points; SWE-bench Pro rises by 5.1 points. Cursor's earlier launch benchmark reported 61.2% against 49% for 4.6, while the current CursorBench version reports effort-specific Sonnet 5 results rather than the old single number. Version drift must be recorded rather than quietly averaged away. *Sources:* [A02], [B14], [P02]

Practitioner evidence adds nuance. CodeRabbit found Sonnet 5 stronger for writing code, persistent enough to keep testing and refining a difficult application without prompting. Yet on its private code-review harness, precision improved from roughly 29% with 4.6 to about 38-40%, while strict bug-catching recall fell: 4.6 found around 63%, Sonnet 5 around 50-51%, and the current production baseline around 57%. Higher effort roughly doubled cost without meaningfully recovering recall. *Sources:* [P06]

Interpretation

Sonnet 5 appears better at building, testing and completing work, but not universally better at every software-engineering subtask. Code review creates a precision-recall trade-off: fewer false alarms can coexist with more missed bugs.

5.3 Tool use and agent persistence: large but expensive

AutomationBench increases from 5.3% to 13.5%, Toolathlon pass@1 from 49.4% to 54.3%, and AA-Briefcase/GDPval-AA place Sonnet 5 around Opus-level professional work. These results support the claim that the model can maintain state, use more tools and keep pursuing a result. They also reveal the price of that behaviour: more turns, more context replay and more opportunities for tool latency. *Sources:* [A02], [B15]

5.4 Reasoning and knowledge work: moderate and uneven

Humanity's Last Exam improves by 8.6 points without tools and 10.6 with tools. HealthBench Professional improves by 13.6 points. Artificial Analysis reports a six-point gain on its composite Intelligence Index, with large improvements on Terminal-Bench, HLE and SciCode but relatively flat results elsewhere. This is meaningful progress, yet not evidence that every reasoning workload improved by the same amount. *Sources:* [A02], [B15]

In professional finance, Anthropic's internal Real-World Finance V2 reports Sonnet 5 at Elo 1219, statistically tied with Opus 4.7 and 4.8 and 219 Elo above 4.6; it wins 69% of direct comparisons with 4.6. The evaluation is useful but internally constructed and model-graded, so it should be treated as strong vendor evidence rather than independent confirmation. *Sources:* [A02]

5.5 Long context: same capacity, different economics

The context window did not increase: both versions advertise 1 million tokens. Sonnet 5 can still be better at using that context, as OfficeQA Full and Pro improve by 4.6 and 6.0 points. However, the new tokenizer can map identical text to more tokens. The effective amount of the same source material that fits into the context may therefore fall, even while the nominal token number remains unchanged. *Sources:* [A02], [A03], [A06]

Long-context evaluation also depends on representation. OfficeQA's own research shows that structured document representations can produce large relative gains compared with raw PDF handling. Anthropic notes that about 9-15% of its OfficeQA episodes hit the output limit and were counted wrong. This is partly a model test and partly a document-ingestion and output-budget test. *Sources:* [A02], [B12]

5.6 Writing and editing: insufficient independent evidence

The launch materials mention professional work and improved instruction following, but there is no strong independent writing benchmark in the evidence reviewed here that isolates long-form prose, editing fidelity, voice preservation and factual restraint. GDPval-AA includes professional deliverables, but it is not a dedicated editorial benchmark. Claims that Sonnet 5 is clearly superior for writing should therefore be marked insufficient evidence rather than inferred from coding performance. *Sources:* [A01], [A02], [B10]

5.7 Computer use, charts and visual documents

OSWorld-Verified rises only 2.7 points, which is small compared with the coding gains. ChartMuseum and CharXiv improve more strongly, especially without tools. BenchCAD is the useful counterexample: no-tool performance is effectively unchanged, while tool-enabled performance rises by 4.6 points. The emerging pattern is that Sonnet 5's advantage often appears when it can act, inspect and revise, rather than when it must solve the task in one unaided response. *Sources:* [A02]

6. Sonnet 5 versus competing models

6.1 Launch-time result: near the frontier, not universal first place

Anthropic's system-card table places Sonnet 5 ahead of GPT-5.5 on SWE-bench Pro, BrowseComp single-agent, both HLE variants, OSWorld, FrontierCode, GDPval-AA, the legal benchmark and HealthBench Professional. GPT-5.5 leads Terminal-Bench 2.1. Gemini 3.5 Flash leads AutomationBench. These are not all apples-to-apples: Anthropic states that competitor figures are drawn from vendor system cards or benchmark leaderboards, and their scaffolds may differ. *Sources:* [A02]

That makes the launch conclusion defensible but narrow: Sonnet 5 was a frontier-class general model with especially strong agentic coding and knowledge work, not a clean sweep. Anthropic's own table contains counterexamples, which is a useful sign that the system card is more informative than a single launch chart. *Sources:* [A02]

6.2 Current independent market: GPT-5.6 changes the answer

By the research cut-off, Artificial Analysis ranks Fable 5 at 60, GPT-5.6 Sol Max at 59, Opus 4.8 Max at 56, Grok 4.5 High at 54 and Sonnet 5 Max at 53 on Intelligence Index v4.1. The index combines nine evaluations, including professional work, terminal use, science, knowledge and long-context reasoning. Sonnet 5 remains close to the top, but it is no longer the strongest current model on this aggregate. *Sources:* [B16], [B17], [P08]

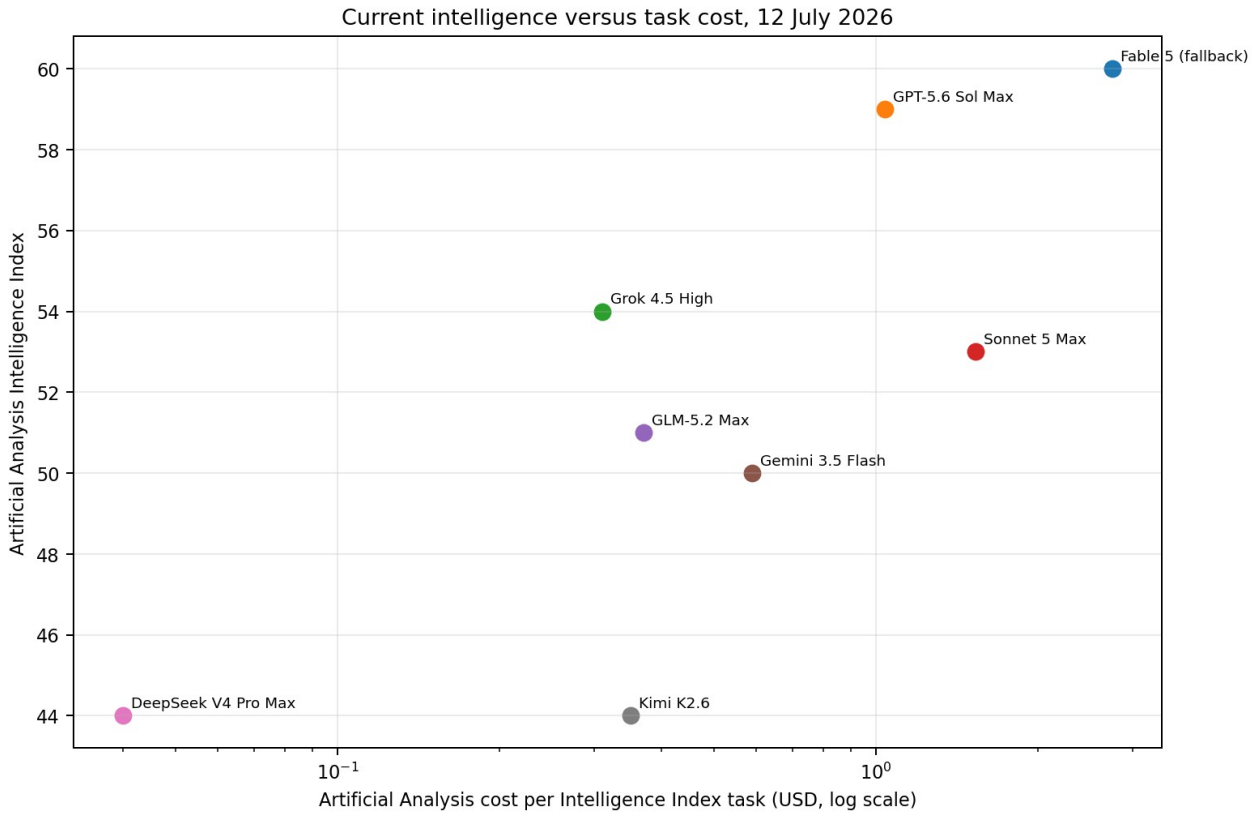

Figure 3. Current intelligence versus reported cost per index task.

Artificial Analysis values are dynamic and should be rechecked before publication. Sonnet 5 is shown at the current promotional-effective figure returned by the comparison page; its launch article also reports $2.29 under standard $3/$15 pricing. Sources: [B15] [B16] [B17].

The price-quality frontier contains different products for different risk tolerances. DeepSeek V4 Pro and Kimi K2.6 are dramatically cheaper but score nine points below Sonnet 5. Gemini 3.5 Flash is three points behind and much faster. Grok 4.5 High is one point ahead at much lower task cost, though Cursor separately discloses a contamination advantage for Grok on its own coding benchmark. No single comparison should be carried across all workloads. *Sources:* [B14], [B16]

6.3 Current coding-agent comparison in CursorBench 3.2

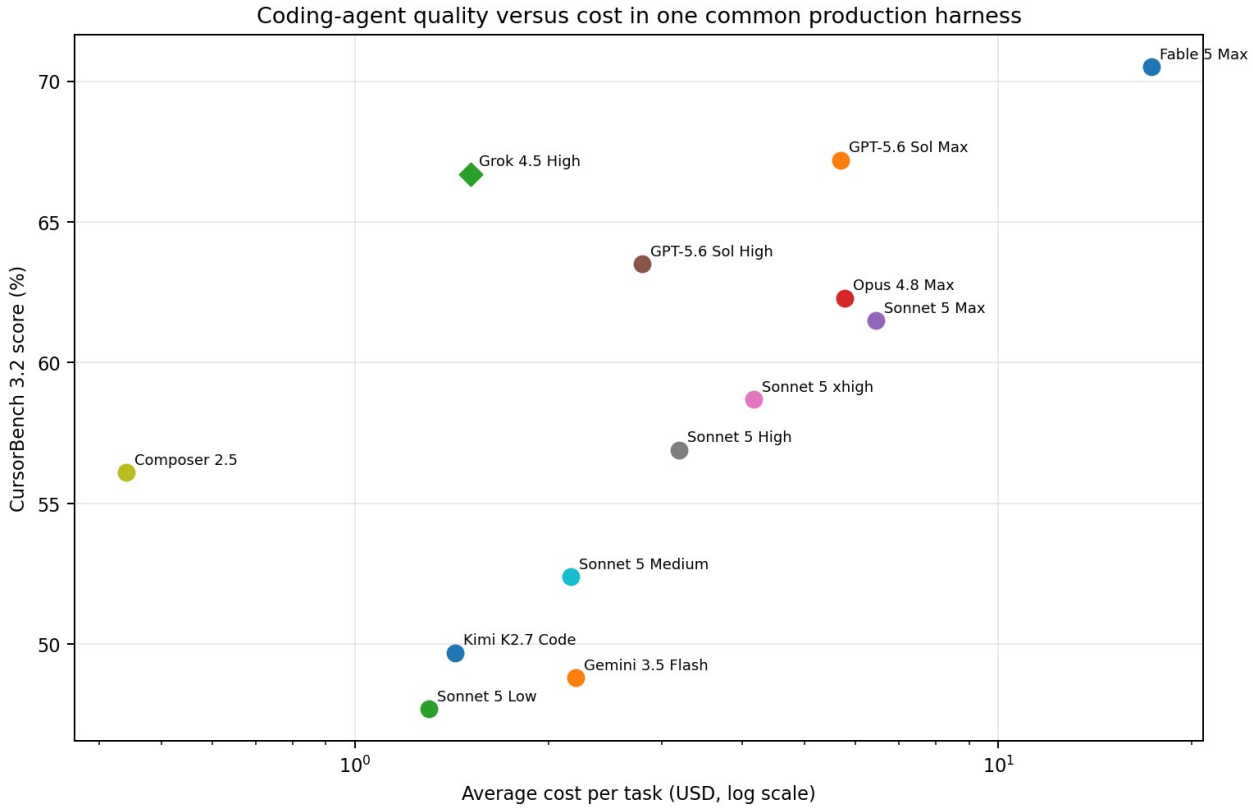

Figure 4. CursorBench 3.2 quality versus average task cost.

The diamond marks Grok 4.5 High, for which Cursor discloses that a prior Cursor code snapshot appeared in training data; the size of the advantage is unknown. Open the interactive source table for tokens and steps: [B14].

Sonnet 5 Max's 61.5% is strong, but it sits below GPT-5.6 Sol High and Max, Grok 4.5 High, Opus 4.8 Max and Fable 5 Max in the current table. Its 86 average steps are also the highest among the listed models. The most attractive Sonnet setting may therefore be medium or high rather than max, depending on the cost of failure. *Sources:* [B14]

CursorBench is not a neutral global referendum. It measures models inside Cursor's production agent, on private tasks drawn from Cursor usage. That is precisely why it is useful for Cursor-like development workflows and why it should not decide which model writes the best essay, diagnoses a data problem or operates another company's tools. *Sources:* [B14]

6.4 Open and lower-cost competitors

DeepSeek V4 Pro, Kimi K2.6 and Qwen-family models occupy a different economic region. On Artificial Analysis, DeepSeek and Kimi reach 44 versus Sonnet 5's 53, while Qwen3.5 397B A17B is lower again. Their appeal is not that they secretly dominate the same frontier workload; it is that many production tasks do not require the last nine index points and can benefit from much lower marginal cost, deployment flexibility or open weights. *Sources:* [B16]

The correct comparison is workload-conditioned. A model that solves 44% of a hard benchmark at one-fortieth the task cost may be excellent for high-volume triage, and terrible for a one-off migration where failure costs a week. Pareto frontiers are more honest than crowns. *Sources:* [B16]

7. Cost per successful task

7.1 Why token price is the wrong denominator

Token tariffs answer a billing question: what does one million tokens cost? A buyer needs an outcome question: what does a correct completed task cost? The gap contains tokenizer behaviour, hidden reasoning, repeated context, prompt-cache misses, tool calls, retries, failed attempts and human clean-up.

Core equation

For a benchmark with average cost c per attempt and independent success probability p, expected cost to the first success is approximately c / p. This is an analytical convenience, not a production guarantee. It assumes retries are independent and identically distributed, ignores learning between attempts, and does not price human review.

7.2 Independent task-level evidence

Artificial Analysis calculates task cost from input, cache write, cache hit, reasoning and answer tokens. At standard pricing, Sonnet 5 Max cost $2.29 per Intelligence Index task, approximately twice Sonnet 4.6 and about 15% more than Opus 4.8. The cause was token consumption rather than a higher list tariff. Promotional pricing reduces the number, but does not remove the behavioural difference. *Sources:* [B15]

A 2026 academic analysis of coding-agent trajectories found that runs on the same task can differ by up to 30 times in total tokens, that higher usage does not reliably imply higher accuracy, and that models systematically underestimate their own likely cost. The paper predates Sonnet 5, but it explains why a single average bill is not enough: production economics need a distribution, not one reassuring mean. *Sources:* [B18]

7.3 CursorBench cost per successful task

Model/setting	Success	Cost/task	Approx. cost/success	Steps	Caveat
Composer 2.5	56.1%	$0.44	$0.78	33	
Grok 4.5 High	66.7%	$1.51	$2.26	33	Contamination advantage disclosed
Sonnet 5 Low	47.7%	$1.30	$2.73	33	
Kimi K2.7 Code	49.7%	$1.43	$2.88	58	
Sonnet 5 Medium	52.4%	$2.16	$4.12	46	
GPT-5.6 Sol High	63.5%	$2.79	$4.39	32	
Gemini 3.5 Flash	48.8%	$2.20	$4.51	77	
Sonnet 5 High	56.9%	$3.19	$5.61	57	
Sonnet 5 xhigh	58.7%	$4.16	$7.09	67	
GPT-5.6 Sol Max	67.2%	$5.69	$8.47	48	
Opus 4.8 Max	62.3%	$5.77	$9.26	44	
Sonnet 5 Max	61.5%	$6.45	$10.49	86	
Fable 5 Max	70.5%	$17.32	$24.57	72	

Within this one benchmark, Sonnet 5's approximate expected cost to a success ranges from $2.73 at low effort to $10.49 at max. The success rate rises from 47.7% to 61.5%, but cost per task rises almost fivefold. GPT-5.6 Sol High reaches a higher score than Sonnet 5 Max with an approximate $4.39 cost per success. Opus 4.8 Max is also slightly cheaper per expected success than Sonnet 5 Max. *Sources:* [B14]

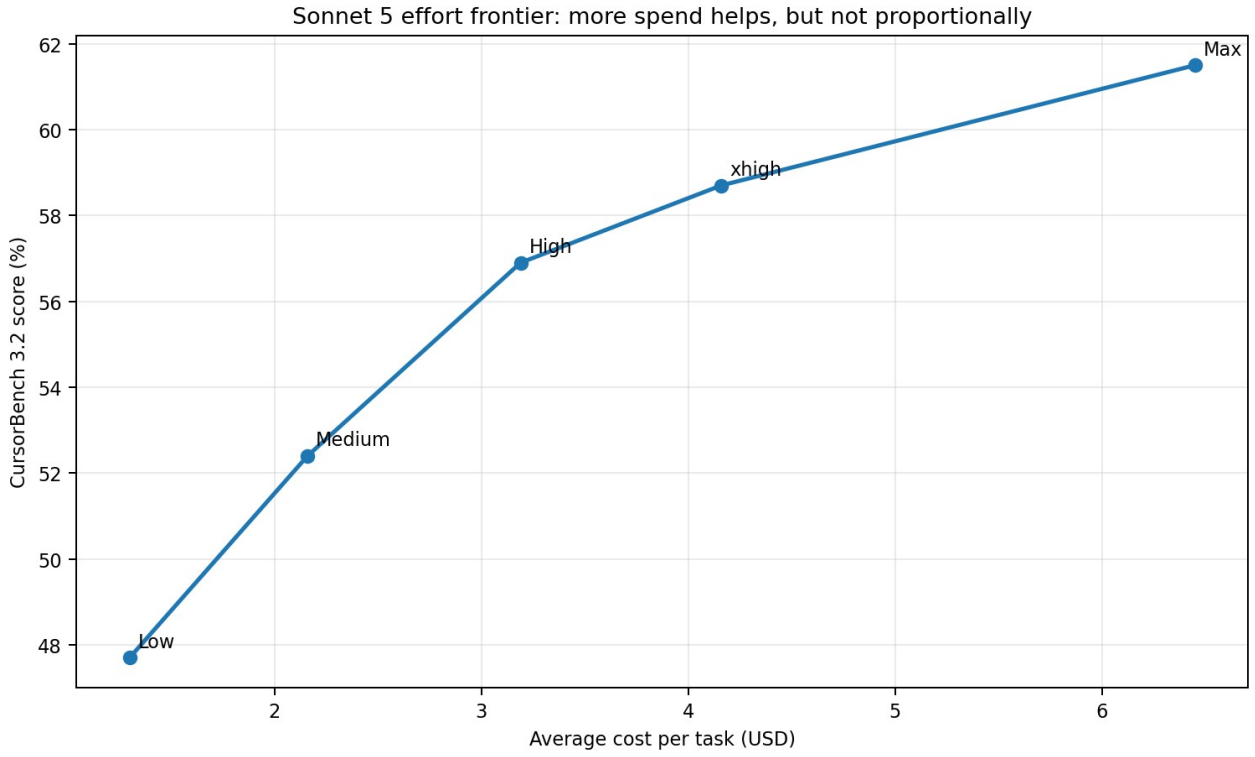

Figure 5. Sonnet 5 effort settings trace a cost-quality frontier rather than a free improvement.

For this workload, max effort buys 4.6 percentage points above high while roughly doubling average task cost. Source: [B14].

7.4 Latency and turns

Latency has at least three layers: time to first token, generation speed and end-to-end task duration. Agentic work is often dominated by the third layer because every model turn can trigger a tool, wait for execution and replay context. Artificial Analysis reports Sonnet 5 as reasonably fast once generating but with very long time to first token at maximum effort. CursorBench reports 57 steps at high and 86 at max, making end-to-end latency a workflow property, not a tokens-per-second property. *Sources:* [B14], [B17]

This leads to a useful operating principle: use the lowest effort that clears the reliability threshold for that task class. Increasing effort globally is convenient, but can spend the budget on easy tasks where extra deliberation has little marginal value.

8. Production and real-world evidence

8.1 Adoption signals

GitHub Copilot made Sonnet 5 generally available on launch day and described strong internal coding results, especially for command-line tasks, alongside good prompt-cache utilisation and competitive latency at lower effort. Databricks also added hosted-model support. These are meaningful adoption signals because platforms do not integrate models for sport, but neither source publishes enough task-level detail to estimate an independent production uplift. *Sources:* [P01], [P03]

8.2 Cursor: quantitative product evidence

Cursor is the strongest public product-style source because it publishes score, average cost, token use and steps under one harness. The evidence confirms that Sonnet 5 is a strong coding-agent model, but also shows that its maximum effort is costly and step-heavy. The current CursorBench 3.2 numbers should replace the initial 57% versus 49% launch figure when discussing today's product; the earlier figure remains useful as historical evidence of a 4.6 delta. *Sources:* [B14], [P02]

8.3 CodeRabbit: improvement and regression in the same workload

CodeRabbit's report is particularly valuable because it does not merely praise the new model. It found stronger code construction, self-testing and cleaner review comments, but slower operation, more tokens, excessive polishing and lower strict bug recall than Sonnet 4.6. In high-volume code review, that means the model may save attention through higher precision while increasing missed defects. Whether that is an upgrade depends on the cost function. *Sources:* [P06]

Why this matters

A model can improve “code quality” and regress “bug finding” because these are different tasks. Product evaluations should preserve that granularity instead of collapsing everything into a coding score.

8.4 Legal and professional workflows

Harvey reports Sonnet 5 at 5.8% all-pass on its held-out Legal Agent Benchmark and 91.2% mean criterion pass, versus 5.4% all-pass for 4.6. The public LAB result rises from 8.0% to 8.9%, yet the system card records a slight decline in mean criterion pass from 88.48% to 88.26%. A stricter all-or-nothing metric can improve while average component quality stays flat or slips. *Sources:* [A02], [P04], [P05]

This is a valuable warning against selecting the friendliest metric. All-pass measures whether a deliverable is completely usable without a single failed criterion; mean criterion pass measures partial quality. Production teams usually need both.

8.5 Security-focused coding evidence

Endor Labs reports Claude Code plus Sonnet 5 at 83.2% functional pass and 19.6% security pass in its Agent Security League. The large gap illustrates a recurrent problem: code can function and still be insecure. This benchmark measures the product configuration, not a bare API model, and therefore belongs in the system-performance column rather than the raw-capability column. *Sources:* [P07]

8.6 Fresh repository evidence from RuBench

RuBench evaluates product configurations on 25 repository tasks written natively in Russian, using fixes that postdate model training cut-offs and three independent runs. The paper reports a best configuration at 78.7%, but correctly notes that the sample is too small to resolve many pairwise differences. Its most important result is methodological: auditing a Fable 5 configuration found that the product silently rerouted five of 25 tasks to Opus 4.8 because of safeguards. *Sources:* [B20]

That observation is direct evidence for Hypothesis 5. When a deployed product can substitute a model, the measured object is the product policy plus agent plus model. A customer may still care only about the product outcome, but a scientific claim about the underlying model must not inherit that result without qualification. *Sources:* [B20]

8.7 Productivity evidence remains incomplete

No large randomised productivity study of Sonnet 5 was available by the cut-off. A prior METR randomised trial using early-2025 AI tools found experienced open-source developers completed assigned tasks 19% more slowly with AI despite expecting acceleration. It is

not evidence against Sonnet 5; it is evidence against translating benchmark gains directly into labour productivity without measurement. *Sources:* [B19]

9. Community evidence

The model had been public for less than two weeks at the research cut-off, so community evidence is necessarily immature. The recurring themes are useful as hypotheses: some users report that low effort guesses instead of grounding through tools; others find medium effort surprisingly expensive; several note that quota use does not feel much lower than Opus; and some prefer Sonnet as a bounded subagent under a stronger planner. *Sources:* [C01]

There is also scepticism about Anthropic's launch-chart revision. Anthropic disclosed that the original BrowseComp cost-performance chart used a simpler methodology and underestimated Sonnet 5, then replaced it with the standard 10-million-token approach. The correction is scientifically preferable to leaving an error in place, but it also demonstrates how strongly the result depends on methodology and how quickly a visual can shape the launch narrative. *Sources:* [A01], [C01]

None of these comments should drive the final verdict. Users vary in task type, context size, subscription accounting, prompting and tolerance for waiting. The strongest community recommendation is almost banal but correct: run the same task several times on your own projects and record quality, spend and latency. The model market has become too harness-dependent for a universal anecdotal ranking. *Sources:* [C01]

10. Contradictions and unresolved questions

Tension	Evidence	Interpretation
“Cheaper than Opus” versus higher task cost	Sonnet 5 has a lower token tariff, yet AA estimates a higher standard-price task cost than Opus 4.8 because it emits and reprocesses more tokens.	Resolved conceptually: tariff and workload cost are different. Exact production economics remain workload-specific.
“Same 1M context” versus less text capacity	The nominal window is unchanged, but the new tokenizer can produce about 30% more tokens for identical text.	Resolved: context must be compared in source bytes or words as well as tokens.
“Better coding” versus lower review recall	Repository-building benchmarks improve, while CodeRabbit reports 4.6 catches more known bugs.	Not a contradiction once coding is split into construction, review precision and review recall.
“Safer against prompt injection” versus successful proof of concept	System-card robustness improves, but an emerging workflow attack reportedly still succeeds.	Expected: risk reduction is not immunity; threat models differ.
“Model result” versus agent result	Most top scores use custom tools and agents. RuBench found model substitution inside a product.	Strongly supports reporting the full system configuration.
“Best at launch” versus not best now	GPT-5.6 arrived nine days later and leads several current comparisons.	A temporal update, not evidence that Sonnet 5 failed to improve.

Evidence still missing

- A large, independent, repeated-run comparison of Sonnet 5 and 4.6 on real company repositories with blinded human review.
- A dedicated writing and editing benchmark measuring factual fidelity, voice preservation, unwanted additions and revision burden.
- Latency distributions, not merely averages, for multi-step agents with identical tools and infrastructure.
- An externally reproducible cost ledger for BrowseComp and other 10M-token evaluations, including cache and tool costs.
- Independent long-context tests that feed the same bytes rather than the same nominal token count across tokenizers.
- Production rollbacks or sustained A/B-test results after several weeks, rather than launch-week testimonials.

11. Who should use Sonnet 5?

User	Recommendation	Reason
Individual developers	Use medium or high first	Best evidence is for building, debugging and multi-file work. Reserve max for tasks where another attempt or human repair is more expensive than the extra tokens.
Claude Code users	Upgrade and evaluate	The model is designed around agentic tool use. Compare on your own repos and log tests passed, retries, steps and wall-clock time.
Software-engineering teams	Selective rollout	Route open-ended implementation and difficult debugging to Sonnet 5; keep latency-sensitive tiny edits on cheaper/faster settings. Evaluate code review separately.
Data scientists	Trial for notebook and repository workflows	Likely useful when analysis includes code execution, data inspection and iterative validation. There is insufficient direct evidence that it is the best statistical analyst.
Agent builders	Strong candidate, instrument heavily	Expose tool-call counts, compaction, cache hit rate, failed loops and safety interventions.

User	Recommendation	Reason
		Model quality and scaffold quality are inseparable.
Enterprises	Adopt behind workload-specific gates	Use governance, prompt-injection controls, least privilege, audit logs and acceptance tests. Do not infer production reliability from one benchmark.
Writing-heavy users	Do not upgrade on benchmarks alone	Run paired blind edits. The research found little independent writing-specific evidence.
Cost-sensitive users	Benchmark medium/low and alternatives	DeepSeek, Kimi, Gemini Flash, Composer or provider-specific models may dominate routine work. Compare cost per accepted output, not token price.
Existing 4.6 deployments	Remain when current pass rate is adequate	Migration changes tokenizer, adaptive thinking and sampling controls. A stable cheap workflow does not become obsolete because a new model is better in another task class.

A practical routing policy

17. Begin with Sonnet 5 medium for complex agentic work and low for bounded routine tasks.
18. Escalate to high only when the expected cost of failure exceeds the measured incremental model cost.
19. Escalate to Opus, Fable, GPT-5.6 or another specialist only for task classes where your eval shows a real gain.
20. Keep Sonnet 4.6 where migration produces no measurable improvement or creates latency/token regressions.
21. Record every run as an experiment: exact model snapshot, effort, prompt, tools, cache, retries, wall-clock latency, spend and acceptance result.

12. Final task-by-task matrix

Task	Sonnet 5 assessment	Strong competitor	Evidence	Cost assessment	Confidence	Important caveats
Coding	Large practical gain over 4.6	GPT-5.6 Sol / Fable 5 in current agent indices	Official SWE/Terminal/FrontierCode; Cursor; CodeRabbit	Strong value at medium/high; max often inefficient	High	Construction and review are different tasks; harness dominates.
Repository-level engineering	Large	GPT-5.6 Sol, Fable 5, Opus 4.8 depending harness	FrontierCode, CursorBench, RuBench	Higher success, but many turns and high context cost	High	Fresh-task samples are small; public tasks risk contamination.
Coding agents	Large	GPT-5.6 Sol current; Opus/Fable for hardest work	CursorBench 3.2 and AA coding/agent indices	Sonnet 5 Max not on current cost frontier	High	Score belongs to agent plus model, not model alone.
Code review	Mixed: precision up, recall down	Workload-specific	CodeRabbit private benchmark	Higher effort doubled cost with little recall gain	Medium	Interested party and private harness; still unusually transparent.
General reasoning	Moderate	Fable 5, GPT-5.6, Opus 4.8	AA Index, HLE, CritPt	Max effort token-heavy	Medium-high	Composite indices hide task variation.
Data analysis	Moderate/inconclusive	GPT-5.6 / Gemini / specialist stacks	Professional benchmarks and tool use, little direct DS evidence	Likely valuable only with code/data tools	Medium-low	Need a dedicated notebook/data-analysis evaluation.
Writing and editing	Insufficient evidence	No defensible universal winner	Broad knowledge-work evidence only	Unknown cost per accepted edit	Low	Run blind editorial tests; do not infer from coding.
Long-context analysis	Moderate quality gain; no capacity gain	GPT-5.6/Opus in current long-context tests	OfficeQA plus API docs	New tokenizer can raise cost and reduce same-text capacity	Medium	Representation, compaction and output limits matter.
Tool use	Large	Fable, GPT-5.6 and Opus often lead	Toolathlon, AutomationBench, BrowseComp	More turns can erase tariff advantage	High	Safety and reliability depend on permissions and tool design.
Computer use	Small-to-moderate	GPT-5.6/Opus depending benchmark version	OSWorld-Verified	End-to-end latency matters more than generation speed	Medium	OSWorld versions and tool bugs changed scores.
Chart/document reasoning	Moderate	Opus often remains ahead	ChartMuseum, CharXiv, OfficeQA	Tool-enabled runs cost more but improve accuracy	Medium	Not equivalent to polished report or slide creation.
Latency	Mixed	Gemini Flash and several smaller models	AA speed; Cursor steps	High/max can be slow despite good output speed	Medium-high	TTFT and full task duration are different.
Reliability	Moderate gain over 4.6	Opus/Fable often stronger	Toolathlon Pass@1/Pass@3/Pass^3	Retries raise expected cost	Medium-high	Need repeated runs on the user's own tasks.
Cost efficiency	Not best by default	GPT-5.6 High, Grok, Gemini, DeepSeek, Kimi or Composer by task	AA and CursorBench	Medium is often the sweet spot; max can be poor value	High	Grok Cursor result has contamination caveat; cheaper models may be weaker.
Enterprise deployment	Strong candidate	Provider/ecosystem dependent	GitHub, Databricks, Harvey, Endor	Caching and batching help; governance costs remain	Medium	Adoption is not proof of uplift. Security controls are mandatory.

13. Reproducible experiments for readers

The experiments below are designed to answer the report's central question on a reader's own workload. Each uses repeated trials, identical scaffolding and explicit economic collection. The intention is not to recreate a public leaderboard, but to estimate a local decision boundary.

Experiment 1: repository-level coding

Field	Specification
Task	Resolve 20-40 closed issues from one or two repositories that resemble the target codebase. Select issues whose fixes postdate the candidate models where possible.
Models	Sonnet 5 medium and high; Sonnet 4.6 high; one current competitor such as GPT-5.6 Sol High; optionally a low-cost model.
Prompt	Use one standard issue prompt: inspect the repository, reproduce the bug, implement the smallest correct fix, add or update tests, run the relevant test suite, and return a concise change summary. Do not expose the hidden patch.
Harness	Use the same container image, shell tools, timeout, memory ceiling, repository state and agent implementation for every model.
Success criteria	Hidden regression tests pass; no unrelated tests regress; patch is within scope; human reviewer marks maintainability and security acceptable.
Scoring rubric	Primary: pass@1. Secondary: pass^3, patch acceptance, files changed, unnecessary churn, security issues and reviewer minutes.
Repetitions	Three independent runs per model-task pair. Randomise model order and reset the environment.
Cost collection	Capture uncached input, cache writes/hits, reasoning/output, tool-provider charges and failed runs. Compute cost per accepted patch and expected cost to first success.
Latency collection	Measure time to first action, model time, tool time and total wall-clock completion. Report median and 90th percentile.
Limitations	Tasks from one repository may favour familiar frameworks. Three runs provide useful reliability information but modest statistical power.

Experiment 2: data-science analysis

Field	Specification
Task	Give each model a messy but bounded analytics problem: a CSV or Parquet dataset, a business question, a data dictionary with omissions and a notebook environment.
Models	Sonnet 5 medium/high, Sonnet 4.6 high, GPT-5.6 or Gemini 3.5 Flash, and one lower-cost model.
Prompt	Inspect the data before making claims. Identify quality issues, define the target metric, conduct the analysis, create one decision-useful chart, quantify uncertainty, and write an executive interpretation. Save executable code and the final artefacts.
Dataset	Use three task types: causal-looking but observational data, a forecasting slice with leakage traps, and an A/B-test dataset with sample-ratio mismatch or multiple testing.
Success criteria	Numerical answers match a hidden reference within tolerance; no leakage; assumptions are stated; code reruns; chart supports rather than decorates the conclusion.
Scoring rubric	Correctness 40%, methodological soundness 25%, reproducibility 15%, communication 10%, efficiency 10%. Apply automatic tests before blinded human review.
Repetitions	Five runs for each of at least six tasks. Data analysis has high path variance, so more repetitions are preferable.
Cost collection	Include Python execution, data uploads, retries and reviewer correction time. Report cost per fully accepted analysis and cost per correct numerical conclusion.
Latency collection	Separate thinking/model time from code execution. Track time to first valid result and time to final accepted deliverable.
Limitations	A small set will not cover all statistics. Model rankings may change sharply between clean tabular work and ambiguous business analysis.

Experiment 3: long-form writing and editing

Field	Specification
Task	Edit six 2,000-4,000 word technical drafts: two explanatory posts, two evidence-led research sections and two pieces with a strict authorial voice.
Models	Sonnet 5 low/medium/high, Sonnet 4.6 high, one OpenAI model and one Google model.
Prompt	Preserve all factual claims and citations. Improve structure and clarity, remove repetition, use British English, do not add unsupported facts,

Field	Specification
	and provide a change log. The full prompt should be identical across models.
Dataset/input	Use unpublished drafts so memorisation is implausible. Seed a known set of subtle problems: duplicated claims, ambiguous antecedents, one unsupported inference and inconsistent terminology.
Success criteria	All seeded issues addressed; no new factual claims; no citation damage; voice-preservation score above threshold; editor accepts with limited rework.
Scoring rubric	Factual fidelity 30%, issue resolution 25%, clarity 20%, voice preservation 15%, unwanted edits 10%. Use blinded pairwise preference plus checklist grading.
Repetitions	Three runs per document and model. Rotate model labels for blinded review.
Cost collection	Count all tokens and editor rework minutes. Convert rework to an internal hourly cost and compute total cost per publishable draft.
Latency collection	Record generation time and human time to acceptance. The latter often dominates the true economics.
Limitations	Editorial preference is partly subjective. Agreement between reviewers should be measured, not assumed.

Experiment 4: effort-level routing

Take 50 representative tasks and run Sonnet 5 low, medium, high and max under identical conditions. Fit an empirical policy that predicts the cheapest effort likely to pass from observable task features: input length, number of files, number of tools, ambiguity, required output length and past failure history. Compare this router with the naive policy of using high or max everywhere. The output should be quality, spend and latency at the portfolio level, not merely a per-task leaderboard. *Sources:* [A05], [B14], [B15]

Appendix A. Complete benchmark claim ledger

Benchmark	Score	Snapshot/date	Reasoning	Tools	Scaffold	Attempts/scoring	Tasks/publicity	Context/budget	Producer	Missing detail/caveat	Sources
SWE-bench Verified	85.2%	S5; 30 Jun 2026	Adaptive thinking, max effort	Repository/shell	Anthropic SWE harness	Average 5 trials	500 public verified issues	1M standard	Anthropic	Exact run-level variance not public	[A02] [B01]
SWE-bench Pro	63.2% vs 58.1% 4.6	S5; 30 Jun 2026	Adaptive, max	Repository/shell	Anthropic standard	Average 5 trials	Harder multi-file issues	1M standard	Anthropic	Competitor harnesses differ	[A02]
SWE-bench Multilingual	78.3%	S5	Adaptive, max	Repository/shell	Anthropic standard	Average 5 trials	300 tasks, 9 languages	1M standard	Anthropic	Limited independent current comparators	[A02]
SWE-bench Multimodal	28.1%	S5	Adaptive, max	Repository + images	Anthropic internal harness	Average 5 trials	Visual issue context	1M standard	Anthropic	Harness details referenced to prior card	[A02]
Terminal-Bench 2.1	80.4% vs 67.0%	S5 xhigh; 4.6 high	xhigh / high	Terminal tools	mini-SWE-agent	5 attempts x 89 tasks	89 tasks after fixes	1x timeout, 3x memory ceiling	Anthropic	Not directly comparable with Terminus-2 results	[A02] [B05]
FrontierCode v1	38.8% vs 15.1%	S5 max	Multiple efforts charted	Coding agent	Cognition benchmark environment	Reported average and cost curve	150 real PR tasks	Not fully specified in summary table	Anthropic using Cognition benchmark	External reproduction limited	[A02]
CursorBench launch	61.2% vs 49%	S5 vs 4.6	Unclear single launch setting	Cursor tools	Cursor production agent	Company aggregate	Real internal/external tasks	Private	Cursor	Superseded by version 3.2 effort table	[A02] [P02]
CursorBench 3.2	61.5% max; 56.9% high	S5 current	Low-max	Cursor tools	Cursor production agent	Published aggregate	Private multi-file tasks	Private	Cursor	No independent task reproduction	[B14]
BrowseComp single	84.7%	S5	Adaptive max	Web/fetch/code/ programmatic tools	Anthropic search agent	System-card result	1,266 hard search questions	10M total; compaction at 200k	Anthropic	Launch chart was revised after methodology correction	[A01] [A02] [B06]
BrowseComp multi	86.6%	S5	Adaptive max	Multi-agent search tools	Anthropic multi-agent	System-card result	Same benchmark	10M total	Anthropic	Not comparable to single-agent cost/latency	[A02]
HLE no tools	43.2% vs 34.6%	S5 vs 4.6	Adaptive max	No web/tools	Direct model evaluation	Average trials	2,500 difficult questions	Up to 1M	Anthropic	Academic closed-answer distribution	[A02] [B08]
HLE with tools	57.4% vs 46.8%	S5 vs 4.6	Adaptive max	Web/fetch/code/ programmatic tools	Anthropic agent	Average trials	HLE with blocklist	1M, no compaction	Anthropic	Tool-rich result is not raw reasoning	[A02] [B08]
OSWorld-Verified	81.2% vs 78.5%	S5 vs re-evaluated 4.6	Max	Computer-use actions	Anthropic computer-use agent	Pass@1 avg 5 runs	361 tasks	100 actions; 128k per turn	Anthropic	4.6 score changed after zoom fix/output increase	[A02] [B07]
ChartMuseum	70.1 no tools; 86.7 tools	S5	Max	Optional Python/tools	Anthropic harness	Reported aggregate	Chart QA	Evaluation-specific	Anthropic	Opus 4.8 remains ahead	[A02]
CharXiv Reasoning	77.0 no tools; 88.3 tools	S5	Max	Optional tools	Anthropic harness	Reported aggregate	Scientific figures	Evaluation-specific	Anthropic	Tool use narrows model differences	[A02]
BenchCAD	26.6 no tools; 37.3 tools	S5	Max	Optional Python	Anthropic harness	Reported aggregate	CAD reasoning	Evaluation-specific	Anthropic	No-tool result flat/slightly below 4.6	[A02]
OfficeQA Full/Pro	73.3 / 59.4	S5	Max	Document search/conversion	Anthropic document agent	Reported aggregate	246 / 133 tasks	300k-1M; output limits matter	Anthropic	9-15% episodes hit output limit	[A02] [B12]
GDPval-AA v2	1609 Elo	S5; Elo as of 6 Jun	Max	Shell + web	Stirrup	Blind pairwise Elo	220 professional tasks	Agent-dependent	Artificial Analysis; included by Anthropic	Model grader and harness influence	[A02] [B10]
Real-World Finance V2	1219 Elo; 69% head-to-head win vs 4.6	S5	Max	Research/analysis tools	Anthropic internal	Pairwise model grade	294 internal tasks	Not fully public	Anthropic	Internal dataset and grader	[A02]
Legal Agent Benchmark public	8.9% all-pass; 88.26% mean criterion	S5	Max	Legal agent tools	Harvey/Anthropic setup	All-pass and criteria	Public legal tasks	Agent-dependent	Anthropic/Harvey	Mean criterion slightly below 4.6	[A02] [P05]
Harvey held-out LAB	5.8% all-pass; 91.2% criterion	S5	Max	Legal tools	Harvey production-style agent	All-pass and criteria	Private held-out	Agent-dependent	Harvey	Interested-party evidence	[A02] [P04]
HealthBench Professional	57.8% vs 44.2%	S5 vs 4.6	Max	Evaluation-specific	Anthropic harness	Reported aggregate	Professional health questions	Evaluation-specific	Anthropic	Not clinical deployment evidence	[A02]
Toolathlon	54.3 Pass@1; 63.0 Pass@3; 40.7 Pass^3	S5	Max	604 tools / 32 apps	Anthropic repaired harness	3 trials; avg 26 turns	108 tasks	Internal dependency pins	Anthropic	~+3 score offset estimated from repairs	[A02] [B11]
AutomationBench	13.5% vs 5.3%	S5 max	Max	Enterprise APIs	Anthropic agent	Deterministic end state	Private held-out	Agent-dependent	Anthropic	Absolute success still low; private tasks	[A02] [B13]
AA-Briefcase	1393 Elo; 183 avg turns	S5	Max	Shell/web	Stirrup	Blind pairwise Elo	Long-horizon knowledge work	Agent-dependent	Artificial Analysis	Many more turns than Opus/Fable	[A02] [B15]

Appendix B. Annotated source register

Each source below is clickable. The confidence tier applies to the kind of claim the source can support. For example, Anthropic's pricing documentation is high-confidence evidence of Anthropic's price, but low-independence evidence that Sonnet 5 is better than a competitor.

Academic paper

[B03] The SWE-Bench Illusion: When State-of-the-Art LLMs Remember Instead of Reason - 1 December 2025 revision. Open source

Evidence: Tier 1 academic evidence. Limitations: Diagnostic evidence of possible memorisation; it does not prove every solved task is contaminated.

[B04] SWE-rebench: continuously refreshed software engineering tasks - 2025. Open source

Evidence: Tier 1 academic evidence. Limitations: Continuous benchmark design is stronger against contamination but still sensitive to harness and infrastructure.

[B06] BrowseComp: A Simple Yet Challenging Benchmark for Browsing Agents - 2025. Open source

Evidence: Tier 1 benchmark paper. Limitations: Hard fact-finding benchmark, not a representative sample of ordinary web research or professional report writing.

[B11] The Tool Decathlon (Toolathlon): multi-turn tool-use benchmark - 2025. Open source

Evidence: Tier 1 benchmark paper. Limitations: Some published tasks were unsatisfiable without dependency pinning; Anthropic reports an internally repaired harness.

[B13] AutomationBench - 2026. Open source

Evidence: Tier 1 benchmark paper. Limitations: Private held-out tasks improve contamination resistance but reduce external reproducibility.

[B18] How Do AI Agents Spend Your Money? - 24 April 2026. Open source

Evidence: Tier 1 academic evidence. Limitations: Predates Sonnet 5; its value is methodological evidence about agent-token variance, not a direct Sonnet 5 ranking.

[B19] Randomised controlled trial of AI coding tools and experienced developers - 2025. Open source

Evidence: Tier 1 academic evidence. Limitations: Uses early-2025 models and tools, not Sonnet 5. It demonstrates that benchmark gains need not translate directly into productivity.

[B20] RuBench 1.0 - 7 July 2026. Open source

Evidence: Tier 1 fresh academic evaluation. Limitations: Only 25 tasks and Russian task specifications; three runs help, but many pairwise gaps are not statistically resolvable.

Anthropic official

[A01] Introducing Claude Sonnet 5 - 30 June 2026. Open source

Evidence: Tier 1 for specifications; vendor evidence for comparative quality. Limitations: Launch framing, selected benchmarks and customer quotations. Not independent validation.

[A02] Claude Sonnet 5 System Card - 30 June 2026. Open source

Evidence: Tier 1 primary documentation. Limitations: Extensive methodology, but most capability results were produced or selected by Anthropic; competitor results may come from different harnesses.

[A03] What's new in Claude Sonnet 5 - Accessed 12 July 2026. Open source

Evidence: Tier 1 product documentation. Limitations: Authoritative for API behaviour and migration details, not comparative model quality.

[A04] Claude API pricing - Accessed 12 July 2026. Open source

Evidence: Tier 1 product documentation. Limitations: List pricing only. Does not represent task-level token use, retries, tools or latency.

[A05] Prompting Claude Sonnet 5 - Accessed 12 July 2026. Open source

Evidence: Tier 1 product documentation. Limitations: Provides intended operating guidance; observed behaviour may vary by workload and agent.

[A06] Claude models overview - Accessed 12 July 2026. Open source

Evidence: Tier 1 product documentation. Limitations: Specifications and availability only.

[A07] Introducing Claude Sonnet 4.6 - 17 February 2026. Open source

Evidence: Tier 1 for predecessor specifications; vendor evidence for quality. Limitations: Useful baseline, but benchmark methods and harnesses changed between releases.

[A08] Demystifying evals for AI agents - Accessed 12 July 2026. Open source

Evidence: Tier 1 methodological guidance. Limitations: General guidance rather than a Sonnet 5 evaluation.

[A09] Infrastructure noise in agentic evaluations - Accessed 12 July 2026. Open source

Evidence: Tier 1 methodological evidence. Limitations: Written by the vendor; nevertheless useful because it explicitly documents benchmark instability.

[A10] SWE-bench and the importance of agent scaffolding - Accessed 12 July 2026. Open source

Evidence: Tier 1 methodological evidence. Limitations: Shows harness sensitivity; it does not independently validate Sonnet 5.

[A11] Claude Sonnet product page - Accessed 12 July 2026. Open source

Evidence: Tier 1 for product positioning; vendor marketing for quality. Limitations: Contains testimonials and claims without full independent methodology.

Benchmark primary source

[B01] SWE-bench Verified - Accessed 12 July 2026. Open source

Evidence: Tier 1 benchmark documentation. Limitations: Static 500-task subset; tasks and repositories are public, so contamination is a material concern.

[B02] SWE-bench repository - Accessed 12 July 2026. Open source

Evidence: Tier 1 code and dataset. Limitations: Reproducibility depends on environment, repository state and agent scaffold.

[B05] Terminal-Bench 2.1 - 2026. Open source

Evidence: Tier 1 benchmark documentation. Limitations: Task fixes between versions mean 2.0 and 2.1 scores are not interchangeable.

[B07] OSWorld - Accessed 12 July 2026. Open source

Evidence: Tier 1 benchmark documentation. Limitations: Computer-use scores are highly sensitive to screen resolution, action budget, UI state and tool bugs.

[B08] Humanity's Last Exam - Accessed 12 July 2026. Open source

Evidence: Tier 1 benchmark documentation. Limitations: Difficult closed-ended academic questions; limited direct evidence for everyday knowledge work.

[B09] GDPval - 2025. Open source

Evidence: Tier 1 benchmark documentation. Limitations: Professional tasks are richer than classic exams, but judgement-based scoring and public subsets introduce their own uncertainty.

[B12] OfficeQA - 2026. Open source

Evidence: Tier 1 dataset and benchmark code. Limitations: Document representation and parsing strategy materially change performance; raw-PDF scores can be much lower.

[P05] Harvey's Legal Agent Benchmark - 2026. Open source

Evidence: Tier 2-3 transparent company benchmark. Limitations: All-pass scoring is stringent but can hide changes in mean criterion quality.

Community evidence

[C01] Hacker News discussion: Claude Sonnet 5 - 30 June-12 July 2026. Open source

Evidence: Tier 4 anecdotal evidence. Limitations: Self-selected anecdotes, unverifiable workloads and strong opinion bias. Useful only for recurring hypotheses.

Engineering report

[P01] Claude Sonnet 5 generally available in GitHub Copilot - 30 June 2026. Open source

Evidence: Tier 3 product evidence. Limitations: Useful deployment signal and qualitative observations; no task-level methodology or numeric results.

[P02] Cursor launch discussion for Claude Sonnet 5 - 30 June 2026. Open source

Evidence: Tier 3 company and community evidence. Limitations: Initial benchmark version differs from current CursorBench 3.2; comments are anecdotal.

[P04] Sonnet 5 in Harvey - 2026. Open source

Evidence: Tier 2-3 interested-party quantitative evidence. Limitations: Harvey evaluates a model used in its own product. Useful numbers, but not an independent neutral comparison.

[P06] CodeRabbit Sonnet 5 review - 30 June 2026. Open source

Evidence: Tier 2-3 quantitative practitioner evaluation. Limitations: Interested party and private harness, but it reports both improvements and regressions with concrete precision/recall figures.

[P07] Endor Labs Agent Security League evaluation - July 2026. Open source

Evidence: Tier 2-3 quantitative security evaluation. Limitations: Measures Claude Code plus Sonnet 5, not the raw model; benchmark scope is security-focused.

Gateway/provider

[P09] Claude Sonnet 5 on OpenRouter - Accessed 12 July 2026. Open source

Evidence: Tier 3 availability and observed provider data. Limitations: Router-level latency and pricing may differ by provider and routing policy.

Independent benchmark organisation

[B10] GDPval-AA methodology - Accessed 12 July 2026. Open source

Evidence: Tier 2 transparent independent evaluation. Limitations: Uses the Stirrup agent harness and model graders; results are agent-system outcomes, not raw-model scores.

[B15] Claude Sonnet 5: strong agentic performance at a higher cost per task - 30 June 2026. Open source

Evidence: Tier 2 strong independent evaluation. Limitations: Artificial Analysis supported Anthropic pre-release. It discloses methodology and costs, but uses its own harnesses and aggregate index.

[B16] Artificial Analysis model leaderboard - Accessed 12 July 2026. Open source

Evidence: Tier 2 independent current leaderboard. Limitations: Dynamic leaderboard; index weights compress heterogeneous tasks into one number.

[B17] GPT-5.6 Sol versus Claude Sonnet 5 comparison - Accessed 12 July 2026. Open source

Evidence: Tier 2 independent evaluation. Limitations: The page is dynamic and the cited comparison mixes reasoning settings unless carefully filtered.

News / emerging security report

[C02] 'Friendly Fire' prompt-injection proof of concept - 10 July 2026. Open source

Evidence: Tier 3 secondary reporting. Limitations: Secondary report about a proof of concept. It shows that improved benchmark robustness does not imply immunity.

OpenAI official

[P08] GPT-5.6: Frontier intelligence that scales with your ambition - 9 July 2026. Open source

Evidence: Tier 1 for GPT-5.6 specifications; vendor evidence for comparisons. Limitations: Released after Sonnet 5. Its vendor benchmark tables should not be treated as independent.

Product release note

[P03] Databricks June 2026 release notes - June 2026. Open source

Evidence: Tier 3 adoption evidence. Limitations: Confirms availability, not superiority or realised production outcomes.

Third-party company benchmark

[B14] CursorBench 3.2 - 8 July 2026 update. Open source

Evidence: Tier 2 quantitative company evaluation. Limitations: Realistic production harness and disclosed cost, but tasks are private and Cursor is an interested party. Grok 4.5 has a disclosed contamination advantage.

Appendix C. Calculation notes and glossary

C.1 Cost calculations

The CursorBench expected cost per success is calculated as average cost per attempt divided by the published success fraction. Example: Sonnet 5 High costs $3.19 per attempt and succeeds on 56.9% of tasks, so $3.19 / 0.569 = approximately $5.61. This is not the cost of a guaranteed solution; it is the expectation under an independent identical-retry model.

Actual retries are rarely independent. A second run may use the same mistaken plan, or a human may improve the prompt after observing the failure. Tool charges and engineering review may also be absent from the benchmark's reported model cost. For production decisions, compute realised total spend divided by accepted outputs over a meaningful period.

C.2 Tokenizer-equivalent context

If identical source text uses 30% more Sonnet 5 tokens, then an old 1,000,000-token payload would use about 1,300,000 new tokens and no longer fit. Conversely, 1,000,000 new tokens correspond to roughly 1,000,000 / 1.30 = 769,231 old-token equivalents. The true ratio varies by content and language, so this is a capacity-planning illustration only. *Sources:* [A03]

C.3 Glossary

Term	Meaning
Agent scaffold	The control system around a model: prompts, tool definitions, file access, loop logic, memory, retries, stopping conditions and error handling.
Adaptive thinking	A mode in which the model dynamically allocates reasoning effort rather than receiving a fixed manual thinking-token budget.
Pass@1	Probability or fraction of tasks solved by the first attempt.
Pass@k	Probability or fraction with at least one successful attempt among k runs; definitions should be checked for the specific benchmark.
Pass^k	Fraction for which all k attempts succeed; a consistency measure.
Elo	Relative rating derived from pairwise comparisons. It is ordinal and depends on the comparison pool and grading method.
Context compaction	Summarising or compressing previous interaction history so an agent can continue beyond the immediate context window.
Cost per successful task	Total evaluation spend divided by the number of accepted or correctly completed tasks.
Contamination	The possibility that benchmark tasks, repository fixes or answers appeared in training data.
Tool call	An invocation of an external capability such as a shell, browser, database, code interpreter or enterprise API.
Time to first token	Delay before generation begins. It is not the same as end-to-end agent completion time.

Closing judgement

Sonnet 5 wins a meaningful contest: it turns the Sonnet tier into a much more capable agentic worker. That matters because many teams can now attempt repository and knowledge-work tasks without immediately paying flagship tariffs. The upgrade is especially real when the job requires the model to inspect, act, test, recover and finish.

It does not win a timeless model championship. Its strongest settings consume enough turns and tokens to lose the cost frontier, its independent non-coding evidence remains limited, and the current market changed within nine days of launch. The scientifically defensible verdict is therefore workload-specific: Sonnet 5 is an excellent candidate, a poor universal default, and a very good reason to measure cost per accepted result rather than admire another launch chart.